

Plan Complet du Cours

Programmation C Avancée

L3 GLSI - Université Cheikh Anta Diop de Dakar

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques

Version 1.0 - Novembre 2025

Contents

1	TP-1: Setup Environnement	3
2	Ch-1: Introduction au Cours	4
3	Ch0: Syntaxe C Complète	5
4	Ch1: Représentation Numérique	8
5	Ch2: Organisation Mémoire	10
6	Ch3: Pointeurs	12
7	Ch4: Tableaux	14
8	Ch5: Chaînes de Caractères	16
9	Ch6: Structures et Unions	18
10	Ch7: Allocation Dynamique	20
11	Ch8: Stack et Calling Conventions	22
12	Ch9: Memory Layout et Cache	24
13	Ch10: POSIX File I/O	26
14	Ch11: Makefiles et Linking	28
15	Ch12: Function Pointers	30
16	Ch13: Preprocessing et Optimisation	32
17	Ch14 (Optionnel): Multithreading Basics	34
18	Ch15 (Optionnel): Networking Basics	35

Vue d'ensemble du cours

Ce cours couvre la programmation C avancée avec une approche orientée systèmes. L'objectif est de former des programmeurs capables de comprendre les mécanismes sous-jacents et d'écrire du code robuste, performant et sécurisé.

Structure pédagogique

- **CM (Cours Magistral):** 8-12 pages, contenu technique dense avec visualisations mémoire TikZ
- **TD (Travaux Dirigés):** 3-5 pages, exercices forensics avec dumps hexadécimaux, prédiction de sorties
- **TD_Corrigé:** Solutions détaillées avec explications pas-à-pas
- **TP (Travaux Pratiques):** 4-6 pages, labs pratiques (compilation, debugging, algorithmes)
- **TP_Pistes:** Indices uniquement, pas de solutions complètes pour les défis algorithmiques

Environnement de travail

Tous les TPs utilisent un environnement Docker standardisé:

Image: `elbachirtoure/c-advanced-course:v1.0`

Outils: `gcc, gdb, valgrind, vim, make`

Référence principale

Computer Systems: A Programmer's Perspective (3rd Edition) par Randal E. Bryant & David R. O'Hallaron

1 TP-1: Setup Environnement

Fichier unique: TP-1_Setup_Environnement.tex

Objectifs

- Installer et configurer Docker
- Maîtriser les commandes Docker de base
- Introduction à vim (modes, commandes essentielles)
- Premier programme C et compilation

Contenu du TP

- Installation Docker (Windows/macOS/Linux)
- Lancement du container et volumes partagés
- Vim: modes (normal, insertion, commande)
- Vim: navigation, édition, sauvegarde
- Exercices vim progressifs
- Premier programme "Hello World"
- Compilation avec gcc
- Exécution du programme

Durée estimée: 2-3 heures

Prérequis: Aucun

Livrables: Environnement fonctionnel + exercices vim complétés

2 Ch-1: Introduction au Cours

Fichier unique: CM_Ch-1_Introduction.tex (2-3 pages maximum)

Objectifs

- Comprendre la pertinence du langage C aujourd'hui
- Connaître l'histoire et l'évolution du C
- Avoir une vue d'ensemble de la structure du cours

Contenu

1. Pourquoi le C?

- Exemples concrets: Linux kernel, PostgreSQL, nginx, FFmpeg
- TensorFlow/PyTorch backends, OpenSSL, Git
- Systèmes embarqués, drivers, OS

2. Histoire du C

- Dennis Ritchie, Bell Labs (1972)
- K&R C, ANSI C (C89), C99, C11, C17, C23
- Influence sur C++, Java, C#, etc.

3. Structure du cours

- Les 6 grandes parties (fondamentaux, représentation, mémoire, structures, exécution, outils)
- Objectifs généraux du cours
- Approche pédagogique (comprendre le "comment ça marche")

4. Prérequis

- Connaissance basique du C (variables, fonctions, boucles)
- Notions d'algorithmique
- Motivation pour la programmation système

5. Environnement de travail

- Docker container standardisé
- Outils: gcc, gdb, valgrind, vim

6. Organisation pédagogique

- CM/TD/TP avec exercices forensics
- Projets pratiques

Note: PAS de section sur l'évaluation, l'intégrité académique, ou les conseils méthodologiques.

3 Ch0: Syntaxe C Complète

Référence: Révision complète des bases du C

CM: Cours Magistral (8-10 pages)

Contenu théorique:

1. Structures de Contrôle

- Blocs et portée (scope)
- if/else, dangling else, opérateur ternaire
- switch et fall-through
- Boucles: while, do-while, for
- break, continue, goto (usage légitime)

2. Types, Opérateurs et Conversions

- Types primitifs et tailles (architecture-dependent)
- Types portables (<stdint.h>)
- Opérateurs: arithmétiques, relationnels, logiques, bit-à-bit
- Priorité et associativité des opérateurs
- Conversions implicites et explicites (cast)
- Pièges: signed/unsigned, promotion entière

3. Déclaration vs Définition

- Différence déclaration/définition
- One Definition Rule (ODR)
- Mots-clés: extern, static
- Erreurs de compilation vs linking

4. Points de Séquence et Comportements Indéfinis

- Définition des points de séquence
- Liste exhaustive des points de séquence
- Undefined Behavior (UB): définition et dangers
- Exemples: modifications multiples, ordre d'évaluation
- Débordement d'entiers signés vs non-signés
- Court-circuit des opérateurs logiques

5. Fonctions et Variables

- Déclaration et définition de fonctions
- Passage par valeur vs simulation passage par référence
- Paramètres const
- Portée: bloc, fonction, fichier, globale
- Classes de stockage: auto, static (local/global), const, volatile
- Récursivité

6. Compilation et Debugging

- Les 4 phases: preprocessing, compilation, assemblage, linking

- Options gcc: -Wall, -Wextra, -Werror, -g, -O0/-O2/-O3
- GDB: commandes essentielles (break, run, next, step, print, backtrace)
- Introduction à Valgrind (aperçu uniquement)

Visualisations:

- Diagrammes de portée (scope)
- Priorité des opérateurs (tableau)
- Phases de compilation (schéma)
- PAS de visualisations mémoire détaillées (Ch2+)

TD: Travaux Dirigés (3-4 pages)

Focus: Prédire la sortie et identifier les erreurs

Exercices:

1. Structures de Contrôle

- Dangling else et ambiguïté
- Shadowing et portée de blocs
- Fall-through dans switch
- Goto légitime (gestion d'erreurs)

2. Priorité et Conversions

- Évaluer expressions complexes
- Piège signed/unsigned
- Promotion entière
- Conversions arithmétiques usuelles

3. Points de Séquence et UB

- Modifications multiples entre points de séquence
- Ordre d'évaluation des arguments
- Débordement d'entiers (signed vs unsigned)

4. Déclaration/Définition

- One Definition Rule (ODR)
- Static vs extern

5. Classes de Stockage

- Static local (conservation entre appels)
- Const et volatile

6. Court-circuit

- Sécurité avec && et ||
- Division par zéro évitée

TD _ Corrigé: Solutions détaillées avec explications

TP: Travaux Pratiques (4-5 pages)

Environnement: Docker (TP-1 complété)

Exercices pratiques:

1. Phases de Compilation

- Preprocessing: gcc -E (examiner .i)
- Compilation: gcc -S (examiner .s assembleur)
- Assemblage: gcc -c (générer .o)
- Linking: gcc (créer exécutable)
- Analyse avec nm et file

2. Options de Compilation

- Compiler avec/sans warnings (-Wall -Wextra)
- Niveaux d'optimisation (-O0, -O2, -O3)
- Comparer tailles et performances

3. Debugging avec GDB

- Compilation avec -g
- Breakpoints, step, next
- Inspection de variables (print)
- Pile d'appels (backtrace)
- Exemple avec fonction récursive

4. Projet Simple

- Calculatrice avec opérations basiques
- Utilise seulement: int, float, boucles, fonctions
- PAS de tableaux, PAS de fichiers, PAS de malloc

TP_Pistes: Indices pour le projet

Durée estimée: 3-4 heures

4 Ch1: Représentation Numérique

Référence: Bryant & O'Hallaron - Chapitre 2

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Systèmes de Numération

- Binaire (base 2), octale (base 8), hexadécimale (base 16)
- Conversions entre bases
- Notation hexadécimale: TOUJOURS 2 caractères par octet (0A, FF, 00)

2. Représentation des Entiers Non-Signés

- Encodage binaire pur
- Plage: $[0, 2^n - 1]$ *Arithmtiquemodulo* 2^n

3. Représentation des Entiers Signés (Complément à Deux)

- Principe du complément à deux
- Plage: $[-2^{(n-1)}, 2^{(n-1)} - 1]$ *Bitdesigne*

- Opérations: négation, addition, soustraction

4. Conversions Signed Unsigned

- Interprétation du pattern de bits
- Pièges et dangers
- Exemples de bugs critiques

5. Extension de Signe

- Extension de 8 bits à 16 bits, 16 à 32, etc.
- Signed vs unsigned extension

6. Représentation IEEE 754 (Flottants)

- Format: signe, exposant, mantisse
- float (32 bits) et double (64 bits)
- Valeurs spéciales: +0, -0, +, -, NaN
- Précision et erreurs d'arrondi
- Pourquoi ne jamais comparer avec ==

7. Dépassements et Débordements

- Overflow/underflow signés (UB)
- Overflow/underflow non-signés (défini)
- Détection et prévention

Visualisations TikZ (5-8 minimum):

- Conversions binaire hex décimal
- Représentation mémoire de valeurs signées/non-signées
- Structure IEEE 754 (décomposition bits)
- Exemples de conversions signed/unsigned avec patterns de bits
- Dumps hexadécimaux avec 2 chars par octet

TD: Travaux Dirigés (4-5 pages)

Focus: Exercices forensics avec dumps hexadécimaux

Exercices:

1. Conversions de Bases

- Binaire $\rightarrow$ Hex $\rightarrow$ Décimal
- Hex $\rightarrow$ Binaire $\rightarrow$ Décimal (signed/unsigned)

2. Complément à Deux

- Calculer la négation
- Identifier la plage pour n bits
- Détecter overflow

3. Forensics Mémoire

- Dumps hex: interpréter comme signed/unsigned
- Prédire les valeurs en mémoire
- Exemples: 0xFF = -1 (signed) ou 255 (unsigned)

4. IEEE 754

- Décomposer float en bits (signe, exposant, mantisse)
- Calculer la valeur décimale
- Identifier NaN, infinités

5. Pièges Conversions

- Prévoir le résultat de conversions signed/unsigned
- Bugs classiques (comparaisons, boucles)

TD _ Corrigé: Solutions avec dumps hex complets (2 chars/octet)

TP: Travaux Pratiques (5-6 pages)

Exercices pratiques:

1. Conversions et Visualisation

- Programmes pour convertir dec/hex/bin
- Afficher représentation mémoire avec dump hex

2. Exploration Signed/Unsigned

- Comparer comportements
- Reproduire bugs classiques

3. IEEE 754 Exploration

- Décomposer float/double en bits
- Tester valeurs spéciales (NaN, infinités)

4. Projet: Bibliothèque de Conversions

- Implémenter convertisseur universel
- Détection d'overflow
- Tests exhaustifs

TP _ Pistes: Indices pour le projet

5 Ch2: Organisation Mémoire

Référence: Bryant Ch.2 (endianness) + minimal assembly

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Modèle Mémoire Linéaire

- Adresses et octets
- Espace d'adressage 32-bit vs 64-bit
- Segments: text, data, bss, heap, stack

2. Endianness

- Little-endian (x86, x86-64)
- Big-endian (anciennes architectures RISC)
- Détection programmatique
- Impact sur le code portable

3. Adressage et Tailles

- `sizeof()` pour types de base
- Architecture-dependent vs portable types
- Adresses: format hexadécimal

4. Introduction Minimale à l'Assembleur

- Registres de base (rax, rbx, rsp, rbp)
- Instructions essentielles: mov, add, sub, lea
- Suffixes: b (byte), w (word), l (long), q (quad)
- Correspondance C $\rightarrow$ assembleur (exemples simples)
- **Note:** PAS un cours complet d'assembleur

5. Représentation en Mémoire

- Variables globales vs locales
- Ordre en mémoire
- Dumps hexadécimaux avec GDB

Visualisations TikZ (6-8):

- Modèle mémoire linéaire (segments)
- Endianness: disposition de 0x12345678 en little/big endian
- Correspondance C/assembleur (exemples mov, add)
- Dumps mémoire hexadécimaux annotés

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. Endianness

- Prédire disposition mémoire little/big endian
- Écrire détecteur d'endianness

2. Forensics Mémoire

- Dumps hex: reconstruire valeurs multi-octets
- Identifier endianness à partir de dump

3. Assembleur Minimal

- Lire code assembleur simple (mov, add)
- Correspondre avec code C

4. Sizeof et Adresses

- Calculer tailles mémoire
- Prédire adresses relatives

TD_Corrigé: Solutions avec dumps hex

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Exploration Endianness

- Détecteur d'endianness
- Visualiser disposition mémoire

2. GDB et Dumps Mémoire

- Commande x/Nx (examiner mémoire)
- Analyser variables en hex

3. Code C → Assembleur

- Compiler avec gcc -S
- Analyser .s généré
- Identifier correspondances

4. Projet: Memory Inspector

- Afficher mémoire en hex
- Interpréter comme différents types

TP_Pistes: Indices projet

6 Ch3: Pointeurs

Référence: Bryant Ch.3 (pointer arithmetic)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Concepts de Base

- Définition: adresse mémoire
- Déclaration: `int *p;`
- Opérateurs: `&` (adresse) et `*` (déréférence)
- NULL et pointeur invalide

2. Arithmétique de Pointeurs

- `p + n` : avance de `n * sizeof(*p)` octets
- `p - q` : nombre d'éléments entre deux pointeurs
- Incrémentation/décrémentation
- Comparaison de pointeurs

3. Pointeurs et Fonctions

- Passage par référence simulé
- Retour de pointeurs (dangers)
- Pointeurs vers fonctions (aperçu)

4. Pointeurs de Pointeurs

- Déclaration: `int **pp;`
- Usage et utilité

5. Pointeurs const

- `const int *p` vs `int * const p`
- `const int * const p`

6. Pièges et Erreurs

- Déréférence de NULL
- Pointeurs pendants (dangling)
- Pointeurs non initialisés
- Aliasing

Visualisations TikZ (6-8):

- Pointeur $\rightarrow$ case mémoire
- Arithmétique: `p`, `p+1`, `p+2` avec adresses
- Pointeurs de pointeurs (double indirection)
- Dumps hex avec pointeurs

TD: Travaux Dirigés (4-5 pages)

Exercices forensics:

1. Arithmétique de Pointeurs

- Calculer $p+n$ avec adresses
- Prédire sorties

2. Dumps Hexadécimaux

- Reconstruire structure de pointeurs
- Suivre indirections

3. Erreurs Classiques

- Identifier bugs (NULL, dangling)
- Corriger code défectueux

TD __ Corrigé: Solutions détaillées

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Manipulation de Base

- Swap via pointeurs
- Fonctions modifiant via pointeurs

2. Arithmétique

- Parcourir zone mémoire
- Calculer distances

3. Debugging

- Traquer erreurs avec GDB
- Valgrind: invalid read/write

4. Projet: Generic Swap

- Swap universel avec `void*`
- Tests sur différents types

TP __ Pistes: Indices

7 Ch4: Tableaux

Référence: Bryant Ch.3 Section 3.8 + CSE 333

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Tableaux 1D

- Déclaration et initialisation
- Accès par indice: `tab[i]`
- Relation avec pointeurs: `tab` & `tab[0]`
- `tab[i]` $\equiv$ `*(tab + i)`
- Décomposition d'adresse

2. Passage de Tableaux aux Fonctions

- Dégénérescence en pointeur
- Nécessité de passer la taille
- Modifications visibles dans l'appelant

3. Tableaux 2D

- Déclaration: `int mat[N][M];`
- Organisation en mémoire (row-major)
- Adressage: `mat[i][j]`
- Arithmétique 2D: `*(*(mat + i) + j)`

4. Tableaux Multidimensionnels

- Tableaux 3D et au-delà
- Calcul d'adresse

5. Pièges

- Dépassement de bornes
- `sizeof(tableau)` vs `sizeof(pointeur)`
- Tableaux de taille variable (VLA)

Visualisations TikZ (6-8):

- Tableau 1D en mémoire avec adresses
- Relation `tab[i]` $\equiv$ `*(tab+i)`
- Tableau 2D: row-major layout
- Dumps hex de tableaux

TD: Travaux Dirigés (4-5 pages)

Exercices forensics:

1. Arithmétique Tableaux

- Prédire adresses
- Calculs avec pointeurs

2. Tableaux 2D

- Layout mémoire
- Accès par arithmétique

3. Dumps Hexadécimaux

- Reconstruire tableau depuis dump
- Identifier dimensions

4. Bugs Classiques

- Dépassement de bornes
- sizeof confusion

TD _Corrigé: Solutions avec dumps

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Manipulation 1D

- Recherche, tri, statistiques
- Deux méthodes: indices et pointeurs

2. Tableaux 2D

- Opérations matricielles simples
- Transposition, addition

3. Projet: Image Processing

- Tableau 2D comme image
- Filtres simples (moyenne, seuillage)

TP _Pistes: Indices

8 Ch5: Chaînes de Caractères

Référence: Bryant Ch.3 + CSE 333

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Représentation

- Tableau de char terminé par `'\0'`
- `char str[]` vs `char *str`
- Chaînes littérales (constantes)

2. Fonctions Standard

- `strlen`, `strcpy`, `strncpy`
- `strcmp`, `strncmp`
- `strcat`, `strncat`
- `strchr`, `strstr`

3. Manipulation Manuelle

- Parcours avec pointeurs
- Modification caractère par caractère

4. Sécurité

- Buffer overflow
- Fonctions dangereuses: `strcpy`, `strcat`, `gets`
- Fonctions sûres: `strncpy`, `strncat`, `fgets`
- Validation d'entrée

5. Conversions

- `atoi`, `atof`, `strtol`, `strtod`
- `sprintf`, `snprintf`

Visualisations TikZ (5-7):

- Chaîne en mémoire avec `'\0'`
- Buffer overflow (avant/après)
- Dumps hex de chaînes

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. Manipulation

- Implémenter `strlen`, `strcpy` manuellement
- Prédire sorties

2. Sécurité

- Identifier buffer overflows
- Corriger code vulnérable

3. Forensics

- Dumps hex de chaînes
- Reconstruire texte

TD_Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Bibliothèque Chaînes

- Réimplémenter fonctions standard
- Tests exhaustifs

2. Parsing

- Tokenisation
- Extraction de données

3. Projet: Text Editor Simple

- Opérations sur texte
- Sécurisé (pas de overflow)

TP_Pistes: Indices

9 Ch6: Structures et Unions

Référence: Bryant Ch.3 Section 3.9

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Structures

- Déclaration, définition
- Initialisation
- Accès membres: . et ->
- typedef

2. Alignment et Padding

- Règles d'alignement (architecture-dependent)
- Calcul de padding
- sizeof(struct) vs somme des membres
- Impact sur performances

3. Structures Imbriquées

- Struct dans struct
- Tableaux de structures
- Pointeurs vers structures

4. Unions

- Définition et usage
- Tous les membres partagent la mémoire
- Cas d'usage

5. Bit Fields

- Déclaration
- Usage (registres hardware, protocoles)
- Portabilité

6. Structures Opaques

- Forward declaration
- Encapsulation

Visualisations TikZ (6-8):

- Structure avec padding (annotation octets)
- Dumps hex de structures
- Union: overlap mémoire
- Alignement: exemples avant/après réorganisation

TD: Travaux Dirigés (4-5 pages)

Exercices forensics:

1. Calcul Padding

- Prédire sizeof(struct)
- Identifier padding

2. Dumps Hexadécimaux

- Reconstruire struct depuis dump
- Identifier alignment

3. Optimisation

- Réorganiser membres pour réduire padding

4. Unions

- Comprendre overlap
- Cas d'usage

TD _ Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Manipulation Structures

- Créer, initialiser, copier
- Tableaux de structures

2. Analyse Mémoire

- GDB: examiner structures
- Dumps hex

3. Projet: Database Simple

- Gestion d'enregistrements
- Recherche, tri

TP _ Pistes: Indices

10 Ch7: Allocation Dynamique

Référence: Bryant Ch.9 (Virtual Memory - malloc/free)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. **Heap et Allocation**
 - Différence stack vs heap
 - malloc, calloc, realloc, free
 - Durée de vie des allocations
2. **Gestion de la Mémoire**
 - Fragmentation interne/externe
 - Stratégies d'allocation (first-fit, best-fit)
 - Coalescing
3. **Erreurs Classiques**
 - Memory leaks
 - Double free
 - Use after free
 - Buffer overflow
 - Pointeurs pendants
4. **Valgrind**
 - Détection de leaks
 - Invalid read/write
 - Utilisation mémoire non initialisée
5. **Allocations Complexes**
 - Tableaux dynamiques 1D
 - Tableaux dynamiques 2D (deux méthodes)
 - Structures dynamiques

Visualisations TikZ (6-8):

- Heap vs stack
- Fragmentation (schémas)
- Erreurs: leak, double free, use after free
- Dumps hex avec metadata malloc

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. **Allocation/Libération**
 - Identifier leaks
 - Corriger erreurs
2. **Tableaux Dynamiques**
 - Allocation 2D
 - Libération correcte
3. **Forensics Valgrind**
 - Interpréter rapports
 - Localiser bugs

TD _Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Détection Erreurs

- Programmes bugués
- Valgrind pour diagnostiquer

2. Structures Dynamiques

- Listes chaînées
- Arbres simples

3. Projet: Dynamic Array Library

- Array redimensionnable
- Tests Valgrind (0 leaks)

TP __ Pistes: Indices

11 Ch8: Stack et Calling Conventions

Référence: Bryant Ch.3 (Machine-Level Programming)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Organisation de la Pile

- Croissance vers adresses basses
- Stack pointer (rsp)
- Base pointer (rbp)

2. Stack Frames

- Structure: paramètres, adresse retour, variables locales
- Prologue et épilogue de fonction
- Frame pointer (optionnel avec -O2)

3. Calling Conventions (x86-64 System V ABI)

- Passage paramètres: registres (rdi, rsi, rdx, rcx, r8, r9) puis stack
- Valeur de retour: rax
- Registres caller-saved vs callee-saved

4. Récursivité

- Stack frames multiples
- Limites et stack overflow

5. Buffer Overflow sur la Stack

- Écrasement adresse de retour
- Protections: stack canaries, ASLR

Visualisations TikZ (6-8):

- Stack frame (détaillé)
- Évolution stack lors d'appels
- Récursivité: frames empilés
- Buffer overflow: avant/après

TD: Travaux Dirigés (4-5 pages)

Exercices forensics:

1. Stack Frames

- Dessiner état stack
- Tracer appels de fonction

2. Calling Conventions

- Identifier registres utilisés
- Paramètres: registres vs stack

3. Dumps GDB

- Examiner stack avec GDB
- Interpréter backtrace

TD _Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Exploration avec GDB

- Examiner frames (info frame)
- Remonter la pile (up/down)

2. Assembleur

- Compiler avec -S
- Identifier prologue/épilogue

3. Projet: Stack Tracer

- Afficher call stack
- Analyser récursivité

TP __ Pistes: Indices

12 Ch9: Memory Layout et Cache

Référence: Bryant Ch.6 (Memory Hierarchy) + Ch.9 (Virtual Memory)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. **Segments Mémoire**
 - Text: code exécutable
 - Data: variables globales initialisées
 - BSS: variables globales non initialisées
 - Heap: allocation dynamique
 - Stack: variables locales, appels
2. **Hiérarchie Mémoire**
 - Registres, L1, L2, L3, RAM, disque
 - Latences et bande passante
 - Principe de localité (temporelle, spatiale)
3. **Cache**
 - Cache lines (typiquement 64 octets)
 - Direct-mapped, set-associative, fully-associative
 - Cache hits vs misses
 - Cold, capacity, conflict misses
4. **Optimisation Cache**
 - Accès séquentiel vs random
 - Blocking/tiling pour matrices
 - Cache-friendly data structures
5. **Mémoire Virtuelle (introduction)**
 - Espace d'adressage virtuel vs physique
 - Pages et page tables
 - TLB (Translation Lookaside Buffer)

Visualisations TikZ (6-8):

- Layout mémoire complet (segments)
- Hiérarchie mémoire (pyramide)
- Cache: structure et fonctionnement
- Accès matrice: row-major vs column-major

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. **Segments**
 - Identifier où variables sont stockées
 - Calculer tailles segments
2. **Cache**
 - Calculer hits/misses pour accès donnés
 - Analyser patterns d'accès
3. **Optimisation**
 - Comparer versions (cache-friendly vs non)

TD _Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Mesure Performances

- Accès séquentiel vs random
- Impact taille cache

2. Optimisation Matrices

- Transposition cache-aware
- Multiplication avec blocking

3. Projet: Cache Simulator

- Simuler cache simple
- Mesurer hits/misses

TP_Pistes: Indices

13 Ch10: POSIX File I/O

Référence: CSE 333 (System calls)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. File Descriptors

- Concept: entiers identifiant fichiers ouverts
- stdin (0), stdout (1), stderr (2)
- Limite par processus

2. System Calls

- open, close
- read, write
- lseek (positionnement)
- Flags: O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_APPEND, etc.
- Permissions (mode)

3. Unbuffered vs Buffered I/O

- System calls: accès direct kernel
- stdio (FILE*): bufferisé
- Performances et cas d'usage

4. Gestion d'Erreurs

- Codes de retour (-1 pour erreurs)
- errno et perror
- strerror

5. Opérations Avancées

- dup, dup2 (duplication descripteurs)
- fcntl (contrôle)
- Redirections

6. Directories

- opendir, readdir, closedir
- stat, fstat

Visualisations TikZ (5-7):

- Table de descripteurs par processus
- Buffering: unbuffered vs buffered
- Redirections avec dup2

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. System Calls

- Écrire programmes simples (copie fichier)
- Gestion d'erreurs

2. File Descriptors

- Tracer état descripteurs
- Comprendre redirections

3. Performances

- Comparer read/write vs fread/fwrite

TD _Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Manipulation Fichiers

- Copie, concaténation
- Statistiques (taille, permissions)

2. Directories

- Lister récursivement
- Filtrer par extension

3. Projet: File Manager

- Mini gestionnaire de fichiers
- Opérations POSIX

TP _Pistes: Indices

14 Ch11: Makefiles et Linking

Référence: CSE 333 (Build systems) + Bryant Ch.7 (Linking)

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Makefiles

- Syntaxe: cibles, dépendances, commandes
- Variables (*CC*, *CFLAGS*, etc.)
- Règles implicites et patterns
- Phony targets (*.PHONY*)
- Makefile modulaire

2. Process de Linking

- Symbol resolution
- Relocation
- Fichiers objets (*.o*)
- Archives statiques (*.a*)

3. Bibliothèques Statiques

- Création avec *ar*
- Linking avec *-l* et *-L*
- Ordre de linking

4. Bibliothèques Dynamiques (Shared Libraries)

- Création (*.so* sous Linux, *.dll* sous Windows)
- Position-Independent Code (*-fPIC*)
- Loading dynamique (*dlopen*, *dlsym*)
- *LD_LIBRARY_PATH*

5. Outils

- *nm*, *objdump*, *ldd*
- Analyse de symboles et dépendances

Visualisations TikZ (5-7):

- Graphe de dépendances Makefile
- Process de linking (schéma)
- Bibliothèques statiques vs dynamiques

TD: Travaux Dirigés (3-4 pages)

Exercices:

1. Makefiles

- Écrire Makefile pour projet
- Optimiser avec règles patterns

2. Linking

- Résoudre erreurs de linking
- Ordre correct bibliothèques

3. Bibliothèques

- Créer .a et .so
- Lier programmes avec

TD __ Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. Makefile Complexe

- Projet multi-fichiers
- Règles de nettoyage

2. Bibliothèques

- Créer bibliothèque statique
- Créer bibliothèque dynamique
- Programmes clients

3. Projet: Build System

- Système de build complet
- Tests, documentation

TP __ Pistes: Indices

15 Ch12: Function Pointers

Référence: Bryant Ch.3 + patterns

CM: Cours Magistral (8-10 pages)

Contenu théorique:

1. Concept

- Pointeurs vers fonctions
- Déclaration: `int (*func_ptr)(int, int);`
- Assignation et appel

2. Typedef pour Lisibilité

- `typedef int (*operation)(int, int);`
- Simplification code

3. Callbacks

- Passer fonction en paramètre
- Exemples: `qsort`, `bsearch`

4. Dispatch Tables

- Tableaux de pointeurs de fonctions
- Switch alternatif
- State machines

5. Applications

- Plugins et extensibilité
- Event handlers
- Algorithmes génériques

Visualisations TikZ (4-6):

- Pointeur de fonction $\rightarrow$ code
- Dispatch table (schéma)
- Callback (flux d'exécution)

TD: Travaux Dirigés (3-4 pages)

Exercices:

1. Déclarations

- Déchiffrer déclarations complexes
- Écrire déclarations

2. Callbacks

- Utiliser `qsort` avec comparateurs
- Écrire comparateurs personnalisés

3. Dispatch Tables

- Implémenter calculatrice
- State machine simple

TD_Corrigé: Solutions

TP: Travaux Pratiques (4-5 pages)

Exercices:

1. Algorithmes Génériques

- Map, filter, reduce
- Tri avec callbacks

2. Event System

- Enregistrement handlers
- Déclenchement événements

3. Projet: Plugin System

- Chargement dynamique
- Interface via function pointers

TP __ Pistes: Indices

16 Ch13: Preprocessing et Optimisation

Référence: Bryant Ch.5 (Optimizing Program Performance) + preprocessing

CM: Cours Magistral (10-12 pages)

Contenu théorique:

1. Preprocessing

- Directives: `#include`, `#define`, `#ifdef`, `#ifndef`, `#endif`
- Macros avec paramètres
- Pièges des macros (effets de bord, priorité)
- Macros vs inline functions
- Conditional compilation
- `#pragma`

2. Include Guards

- Prévenir inclusions multiples
- `#pragma once` (non-standard mais répandu)

3. Optimisations Compilateur

- Niveaux: `-O0`, `-O1`, `-O2`, `-O3`, `-Os`
- Optimisations courantes: inlining, loop unrolling, constant folding
- Profile-guided optimization (PGO)

4. Optimisations Manuel

- Réduction appels de fonction
- Loop hoisting
- Strength reduction
- Cache-aware code

5. Inline Functions

- Keyword `inline`
- Avantages vs macros
- Limitations

6. Profiling

- `gprof`
- `perf` (Linux)
- Identifier hotspots

7. Trade-offs

- Performance vs lisibilité
- Space vs time
- Premature optimization

Visualisations TikZ (5-7):

- Expansion de macros
- Comparaison code non-optimisé vs optimisé
- Impact cache (before/after optimization)

TD: Travaux Dirigés (4-5 pages)

Exercices:

1. **Macros**
 - Écrire macros correctes
 - Identifier bugs dans macros
2. **Preprocessing**
 - Expansion manuelle
 - Conditional compilation
3. **Optimisations**
 - Identifier opportunités
 - Appliquer transformations

TD _ Corrigé: Solutions

TP: Travaux Pratiques (5-6 pages)

Exercices:

1. **Analyse Preprocessing**
 - Examiner .i files
 - Comprendre expansions
2. **Profiling**
 - Utiliser gprof
 - Identifier hotspots
3. **Optimisation**
 - Programme lent fourni
 - Optimiser (manuel + compilateur)
 - Mesurer gains
4. **Projet: Performance Challenge**
 - Implémenter algorithme
 - Optimiser au maximum
 - Benchmark

TP _ Pistes: Indices

17 Ch14 (Optionnel): Multithreading Basics

Référence: Bryant Ch.12 (Concurrent Programming) - introduction seulement

CM: Cours Magistral (8-10 pages)

Contenu théorique:

- Threads vs processus
- `pthread_create`, `pthread_join`
- Problèmes: race conditions, deadlocks
- Mutexes (`pthread_mutex_t`)
- Introduction synchronisation

Note: Cours d'introduction uniquement, pas de traitement exhaustif.

TD & TP: (3-4 pages chacun)

Exercices simples de création de threads et synchronisation basique.

18 Ch15 (Optionnel): Networking Basics

Référence: Bryant Ch.11 (Network Programming) - introduction seulement

CM: Cours Magistral (8-10 pages)

Contenu théorique:

- Modèle client-serveur
- Sockets: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`
- TCP vs UDP (aperçu)
- Exemple simple: echo server

Note: Cours d'introduction uniquement, pas de traitement exhaustif.

TD & TP: (3-4 pages chacun)

Exercices simples de communication réseau.

Résumé de la Structure du Cours

Chapitres Obligatoires

- **TP-1:** Setup environnement Docker + vim
- **Ch-1:** Introduction (2-3 pages)
- **Ch0:** Syntaxe C complète (fondamentaux)
- **Ch1:** Représentation numérique
- **Ch2:** Organisation mémoire
- **Ch3:** Pointeurs
- **Ch4:** Tableaux
- **Ch5:** Chaînes
- **Ch6:** Structures et unions
- **Ch7:** Allocation dynamique
- **Ch8:** Stack et calling conventions
- **Ch9:** Memory layout et cache
- **Ch10:** POSIX File I/O
- **Ch11:** Makefiles et linking
- **Ch12:** Function pointers
- **Ch13:** Preprocessing et optimisation

Chapitres Optionnels

- **Ch14:** Multithreading (introduction)
- **Ch15:** Networking (introduction)

Volume Total

- **CM:** 13 chapitres $\times$ 10 pages 130 pages
- **TD:** 13 chapitres $\times$ 4 pages 52 pages
- **TD_Corrigé:** 13 chapitres $\times$ 4 pages 52 pages
- **TP:** 13 chapitres $\times$ 5 pages 65 pages
- **TP_Pistes:** 13 chapitres $\times$ 2 pages 26 pages
- **TOTAL:** 325 pages de contenu technique dense

Fichiers à Générer (hors optionnels)

- 1 fichier TP-1
- 1 fichier Ch-1 (Introduction)
- Pour Ch0-Ch13 (14 chapitres): 5 fichiers chacun
- **Total:** $1 + 1 + (14 \times 5) = \mathbf{72}$ fichiers